

使用 Actions Builder 為 Google 助理建立 Interactive Canvas 動作

來源：<https://codelabs.developers.google.com/codelabs/builder-canvas?hl=zh-tw>

步驟數：11

瞭解如何為 Google 助理建立互動式畫布動作。

目錄

1. 總覽
2. [Interactive Canvas 簡介](#)
3. [設定](#)
4. [設定 Actions 專案](#)
5. [瞭解互動式畫布基礎架構](#)
6. [新增猜測功能](#)
7. [新增成交/未成交功能](#)
8. [新增重新播放功能](#)
9. [更新 PLAY_GAME 內建意圖](#)
10. [附錄：排解 Interactive Canvas 動作問題](#)
11. 恭喜！

1. 總覽

警告：對話動作已於 2023 年 6 月 13 日淘汰。詳情請參閱 [Conversational Actions 終止支援](#)。

Actions on Google 是開發人員平台，可讓您建立軟體來擴充 [Google 助理](#) 的功能。Google 助理是虛擬個人助理，支援超過 10 億部裝置，包括智慧音箱、手機、汽車、電視、耳機等。使用者可以透過對話與 Google 助理互動，完成購物或預約車輛等工作。如需目前可執行的動作完整清單，請參閱[動作目錄](#)。開發人員可以使用 Actions on Google，輕鬆建立及管理使用者與第三方服務之間，令人愉悅且實用的對話體驗。

本程式碼研究室是進階模組，適合已具備 Google 助理 Actions 建構經驗的讀者。如果您沒有使用 Actions on Google 開發的經驗，請先完成入門 Codelab ([第 1 級](#)和[第 2 級](#))，熟悉這個平台。這些模組將引導您瞭解一系列功能，協助您擴展動作的功能並增加目標對象。

在本程式碼研究室中，您將使用以 Google 助理為基礎建構的 [Interactive Canvas](#) 架構，在對話動作中加入全螢幕遊戲。這款遊戲是互動式網頁應用程式，Google 助理會在對話中傳送給使用者。使用者接著就能在智慧螢幕和 Android 行動裝置上，透過語音或文字輸入玩遊戲。

您不必自行建構整個遊戲，而是部署名為「Snow Pal」的部分預先建構遊戲，並在完成程式碼研究室的過程中新增遊戲編輯。[雪人朋友](#)是傳統[猜單字](#)遊戲的變體。遊戲會顯示代表單字的空白格，您要猜測單字中可能包含的字母。每猜錯一次，雪人就會融化一點。如果能在雪人完全融化前猜出單字，就能贏得遊戲。

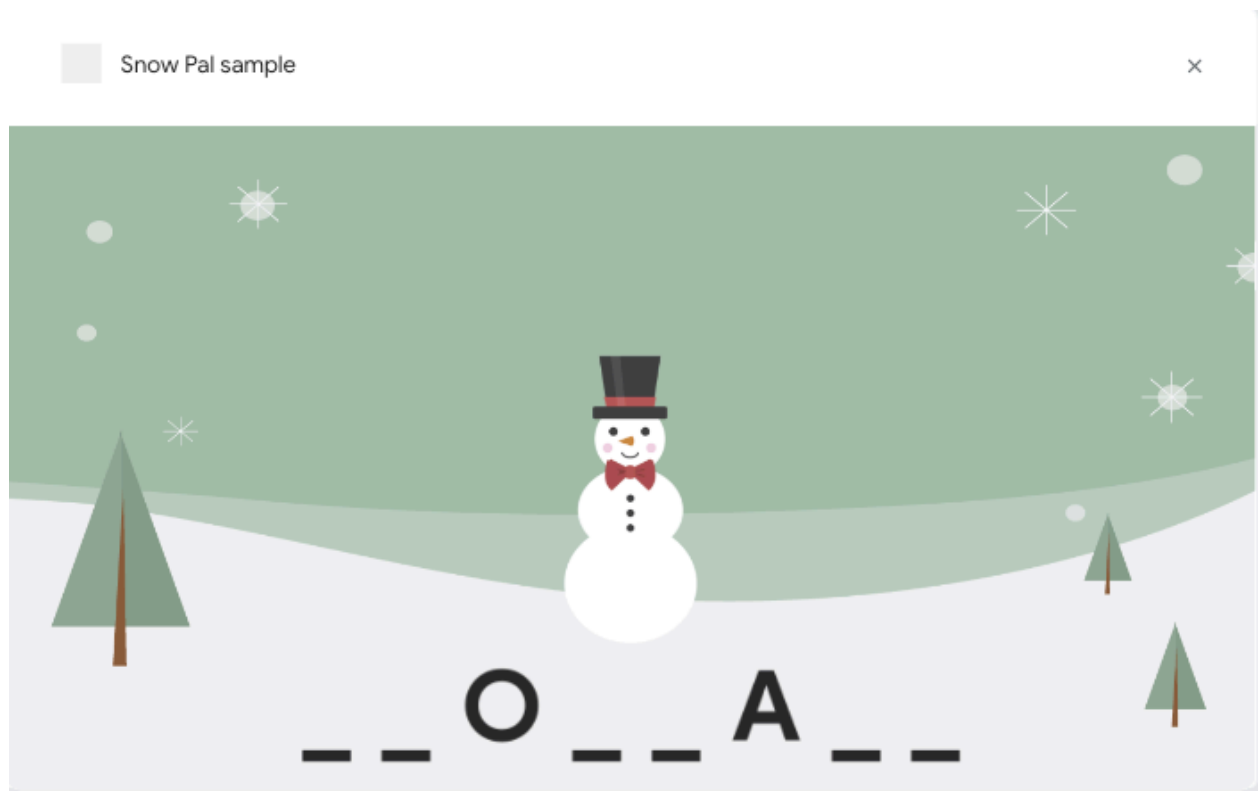


圖 1. 部分完成的 Snow Pal 遊戲

建構項目

在本程式碼研究室中，您將建構使用互動式畫布的動作。你的動作將具備下列功能：

- 使用者可透過語音指令操作的全螢幕單字遊戲
- 使用者可按下按鈕開始遊戲

- 使用者可按下按鈕再次玩遊戲

完成本程式碼研究室後，您完成的動作會包含下列對話流程：

Google 助理： Welcome to Snow Pal! Would you like to start playing the game?

使用者： Yes.

Google 助理： Try guessing a letter in the word or guessing the word.

使用者： I guess the letter E .

Google 助理： Let's see if your guess is there...E is right. Right on! Good guess.

使用者會繼續猜測字母或未知單字本身，直到遊戲結束為止。

課程內容

- 如何建構及部署 Interactive Canvas 動作
- 如何根據使用者的語音和觸控輸入更新文字遊戲
- 如何使用不同方法將資料傳遞至網頁應用程式
- 如何偵錯 Interactive Canvas 動作

軟硬體需求

本程式碼研究室的先決條件包括：

- 網路瀏覽器，例如 [Google Chrome](#)
- 您選擇的 IDE 或文字編輯器
- 電腦上已安裝 [Node.js](#)、[npm](#) 和 [git](#)

強烈建議您熟悉 [JavaScript](#) (ES6)，以便瞭解本程式碼研究室使用的程式碼。

選用：取得完整範例程式碼

在本程式碼研究室中，您會從不完整的版本逐步建構範例。您也可以視需要從 [GitHub 存放區](#) 取得完成的範例。

2. Interactive Canvas 簡介

互動式畫布是以 Google 助理為基礎建構的架構，可讓您在對話式動作中加入全螢幕視覺元素和動畫。

重要字詞：

- **對話動作**：使用對話介面滿足使用者要求的動作。

使用 Interactive Canvas 的動作與一般對話**動作**類似，不過，互動式畫布動作還能將回應傳送至開啟全螢幕網頁應用程式的裝置。

重要字詞：

- **網頁應用程式**：使用 HTML、CSS 和 JavaScript 等網頁技術建立的應用程式，可提供互動式視覺體驗，例如遊戲。在本程式碼研究室中，網頁應用程式是 *Snow Pal* 遊戲的視覺效果和動畫。

使用者透過語音或觸控方式輸入內容，直到對話結束為止。



圖 2. 使用 Interactive Canvas 建構的動作。

在本程式碼研究室中，專案分為下列三個主要部分：

- **網頁應用程式**：網頁應用程式檔案包含網頁應用程式的視覺效果和動畫程式碼，以及根據使用者輸入內容更新網頁應用程式的邏輯。
- **對話動作**：對話動作包含對話邏輯，可辨識、處理及回應使用者的輸入內容。
- **Webhook**：Webhook 會處理遊戲邏輯。在本程式碼研究室中，您可以將 Webhook 視為遊戲伺服器。

重要字詞：

- **Webhook**：處理場景或意圖要求的 HTTP 回呼，通常用於動態建構對話回應 (通常是透過發出 API 呼叫)。

在本程式碼研究室的每個部分，您都會修改網頁應用程式、對話式動作和 Webhook，為互動式畫布動作新增功能。

大致來說，*Snow Pal* 動作中的對話動作、Webhook 和網頁應用程式會以以下方式運作：

1. 對話動作會提示使用者猜測字詞中的字母或整個字詞。
2. 使用者在智慧螢幕上對 *Snow Pal* 網頁應用程式說出「我猜是字母 i」。
3. 系統會將使用者輸入內容傳送至您在 Actions Builder 和/或 Actions SDK 專案中定義的對話動作。
4. 對話動作會處理使用者的輸入內容，並視輸入內容而定，觸發 Webhook 中的邏輯來更新網頁應用程式，或直接傳送中繼資料來更新網頁應用程式。
5. 網頁應用程式會更新，顯示字母在字詞中的位置，並要求使用者再次猜測。

您可以在「[瞭解互動式畫布基礎架構](#)」一節中，進一步瞭解互動式畫布的運作方式。您現在已掌握基本知識，可以開始為本程式碼研究室設定環境。

3. 設定

檢查 Google 權限設定

若要測試為本程式碼實驗室建構的動作，您必須啟用必要權限，讓 Actions 控制台模擬器存取您的動作。如要啟用權限，請按照下列步驟操作：

1. 前往「[活動控制項](#)」。
2. 視需要登入 Google 帳戶。
3. 啟用下列權限：
 - **網路和應用程式活動**
 - 在「網路和應用程式活動」下方，勾選「包括 Chrome 歷史記錄，以及採用 Google 服務的網站、應用程式和裝置上的活動記錄」核取方塊。

安裝 gactions 指令列介面

在本程式碼研究室中，您將使用 *gactions* 指令列介面 (CLI) 工具，在 Actions 控制台和本機檔案系統之間同步處理 Actions 專案。

關鍵字詞：

- **gactions CLI**：這項工具可讓您初始化動作專案檔案、將更新推送至 Actions 控制台，以及部署動作專案等。

如要安裝 gactions CLI，請按照「[安裝 gactions 指令列工具](#)」一節中的操作說明進行。

安裝 Firebase 指令列介面

Firebase 指令列介面 (CLI) 可讓您將 Actions 專案部署至 Cloud Functions，並代管網頁應用程式。

如果您已安裝 [Firebase CLI](#)，可以略過這些步驟，直接前往下一節。

本程式碼研究室會使用 [npm](#) 安裝 Firebase CLI。請務必安裝 npm，這通常會隨附於 [Node.js](#)。

1. 如要安裝或升級 CLI，請開啟終端機並執行下列 `npm` 指令：

```
npm install -g firebase-tools
```

無法順利完成遷移嗎？您可能需要[變更 npm 權限](#)。

2. 如要確認 CLI 是否已正確安裝，請執行下列指令：

```
firebase --version
```

請確認 Firebase CLI 版本為 8 以上，以便使用 Cloud Functions 的所有最新功能。如果不是，請執行 `npm install -g firebase-tools` 升級。

3. 如要登入 Firebase，請執行下列指令：

```
firebase login
```

登入 Firebase 時，請務必使用建立動作專案時的 Google 帳戶。

複製存放區

在本節中，您會複製本程式碼研究室所需的檔案。如要取得這些檔案，請按照下列步驟操作：

1. 開啟終端機，然後變更為您通常儲存程式碼專案的目錄。如果沒有，請變更為主目錄。
2. 如要複製這個存放區，請在終端機中執行下列指令：

```
git clone https://github.com/actions-on-google/actions-builder-canvas-codelab-nodejs
```

開啟 `start/` 目錄。這個存放區包含下列重要目錄，您將會使用這些目錄：

- `public/`：這個目錄包含網頁應用程式的 HTML、CSS 和 JavaScript 程式碼。
- `sdk/custom/`：這個目錄包含對話動作的邏輯 (場景、意圖和型別)。
- `sdk/webhooks/`：這個目錄是你的 Webhook，內含遊戲邏輯。

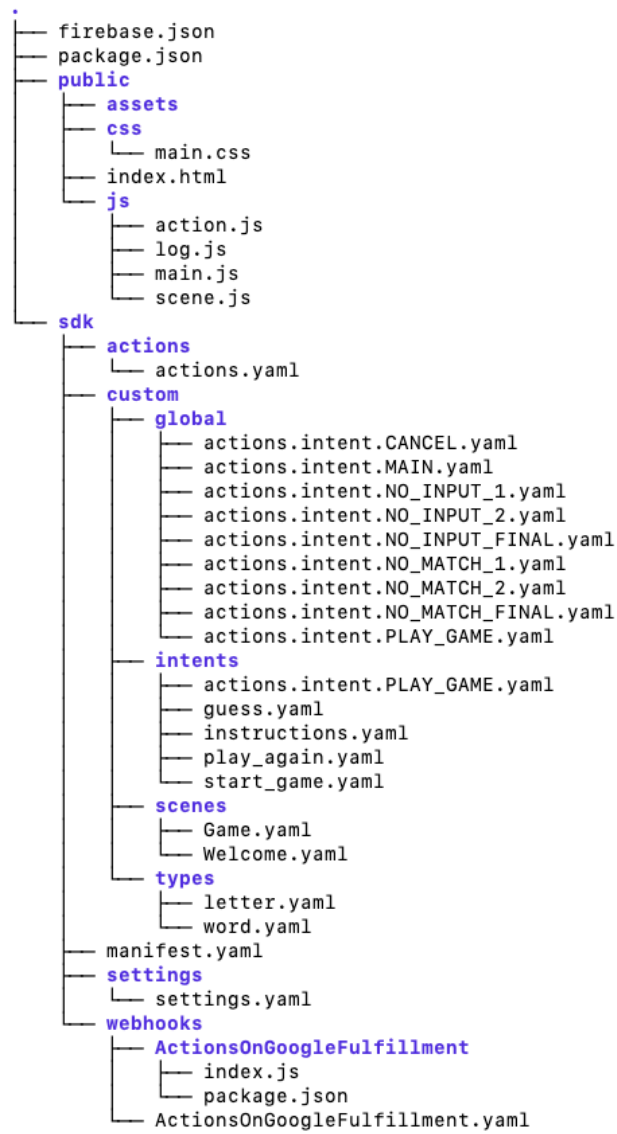


圖 3. start 目錄程式碼的結構。

注意：完成的程式碼檔案位於 `complete/` 目錄中。

步驟 4 · 約 0 分鐘

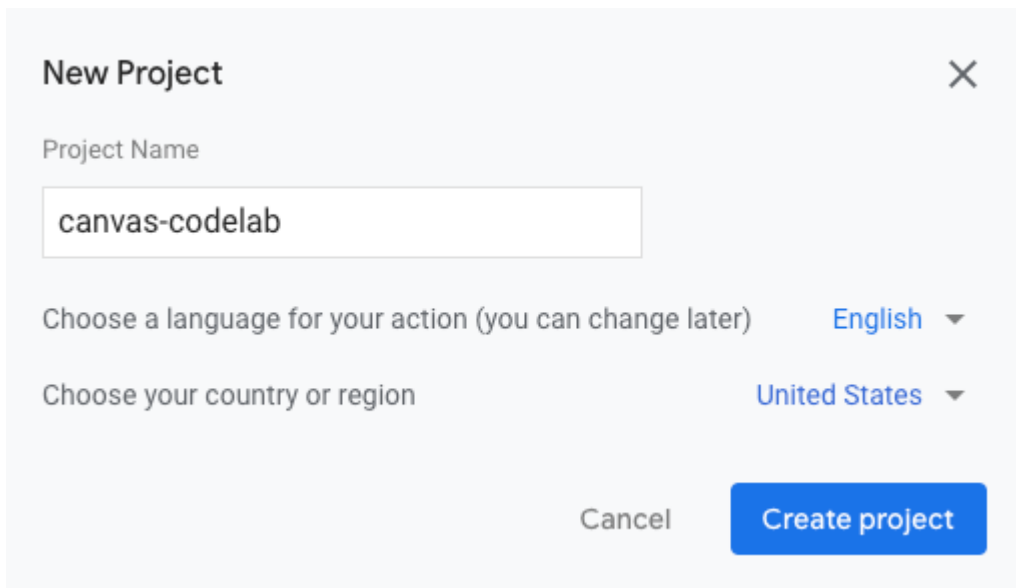
4. 設定 Actions 專案

在本節中，您將建立及設定 Actions 專案，使用 gactions CLI 將複製的存放區中的程式碼推送至 Actions 控制台，並部署網頁應用程式和 Webhook。

建立 Actions 專案

動作專案是動作的容器，如要為本程式碼研究室建立 Actions 專案，請按照下列步驟操作：

1. 開啟 [Actions 管理中心](#)。
2. 按一下 [新專案]。
3. 輸入專案名稱，例如 `canvas-codelab`。這個名稱僅供內部參照，稍後，您可以為專案設定外部名稱。



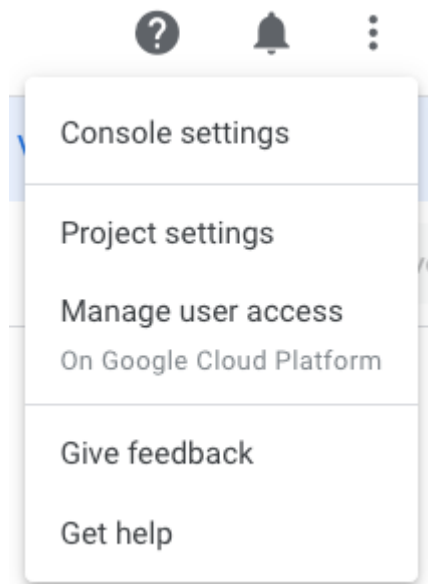
4. 按一下 [Create Project]。
5. 在「您想建構哪種動作？」畫面上，選取「遊戲」資訊卡。
6. 點選 [下一步]。
7. 選取「空白專案」資訊卡。
8. 按一下「開始建構」。

儲存動作的專案 ID

專案 ID 是動作的專屬 ID。本程式碼研究室的幾個步驟會用到專案 ID。

如要擷取專案 ID，請按照下列步驟操作：

1. 在 Actions 控制台中，依序點選三個垂直圓點 > 「專案設定」。



2. 複製專案 ID。

建立帳單帳戶關聯

如要在本程式碼研究室稍後使用 Cloud Functions 部署完成的動作，請務必將帳單帳戶與 Google Cloud 中的專案建立關聯。如果您已有帳單帳戶，可以略過下列步驟。

如要將帳單帳戶與專案建立關聯，請按照下列步驟操作：

1. 前往 [Google Cloud Platform 帳單頁面](#)。
2. 按一下「新增帳單帳戶」或「建立帳戶」。
3. 填寫付款資訊。
4. 按一下「開始免費試用」或「提交資訊並啟用計費功能」。
5. 前往 [Google Cloud 帳單頁面](#)。
6. 按一下「我的專案」分頁標籤。
7. 依序點選三點圖示 > 「變更帳單」。
8. 在下拉式選單中，選取您設定的帳單帳戶。
9. 按一下 [設定帳戶]。

如要避免產生費用，請按照「[清除專案](#)」一節的步驟操作。

部署網頁應用程式

在本節中，您將使用 Firebase CLI 部署網頁應用程式 (*Snow Pal* 遊戲)。部署完成後，您就能擷取網頁應用程式的網址，並在瀏覽器中查看遊戲。

您可以選擇定義動作，允許使用 Firebase 指令 `firebase deploy` 部署實現檔案和網頁應用程式檔案。在本程式碼研究室中，您將分別部署網頁應用程式和完成要求。

如要部署網頁應用程式，請按照下列步驟操作：

1. 在終端機中，前往 `start/` 目錄。
2. 執行下列指令，將 `{PROJECT_ID}` 替換為您的專案 ID：

```
firebase deploy --project {PROJECT_ID} --only hosting
```

幾分鐘後，您應該會看到 " **Deploy complete!** "，表示您已成功將 Snow Pal 網頁應用程式部署至 Firebase。

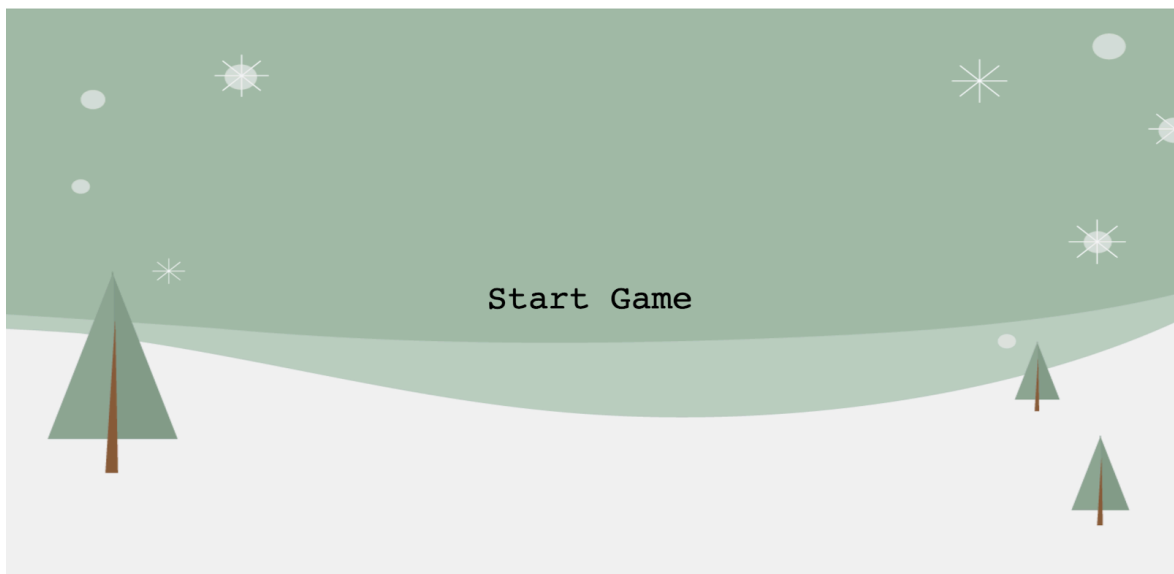
如要在瀏覽器中查看「Snow Pal」遊戲，請按照下列步驟操作：

1. 從終端機的輸出內容中，擷取提供的代管網址。網址格式應為 `https://<PROJECT_ID>.web.app`

如果網址未顯示在終端機輸出內容中，請按照下列步驟擷取網頁應用程式網址：

1. 開啟 [Firebase 控制台](#)。
2. 從選項清單中選取 Actions 專案。
3. 按一下導覽列中的「開發」>「託管」。
4. 在「資訊主頁」分頁下方，您應該會看到專案的網域清單。複製網址。格式應為：
`https://<PROJECT_ID>.web.app`

2. 在瀏覽器中貼上網址。畫面上應會顯示「Snow Pal」遊戲的開始畫面，以及「Start Game」按鈕：



將網頁應用程式網址和專案 ID 新增至 Actions 專案

接著，請將網頁應用程式網址和專案 ID 新增至 `actions.intent.MAIN.yaml` 檔案。新增網頁應用程式網址可讓對話動作知道要將資料傳送至哪個網址，而在 `settings.yaml` 中新增專案 ID，則可將下載的檔案推送至 Actions 控制台中的正確專案。

如要新增網頁應用程式網址和專案 ID，請按照下列步驟操作：

1. 在文字編輯器中開啟 `start/sdk/custom/global/actions.intent.MAIN.yaml` 檔案。
2. 在「網址」欄位中，將預留位置字串替換為您的網頁應用程式網址。
3. 在文字編輯器中開啟 `start/sdk/settings/settings.yaml` 檔案。
4. 在 `projectId` 欄位中，將預留位置字串替換為您的專案 ID。

將專案推送至 Actions 主控台

如要使用本機專案更新 Actions 主控台，請將 Actions 專案推送至 Actions 主控台。步驟如下：

1. 切換至 `sdk` 目錄：

```
cd sdk/
```

2. 如要將本機檔案系統的設定複製到 Actions 控制台，請執行下列指令：

```
gactions push
```

部署 Webhook

您在執行 `gactions push` 時，已將起始 Webhook 程式碼匯入 Actions 管理中心。在本程式碼研究室的其餘部分，您可以在 Actions 控制台中編輯 Webhook 程式碼。此時，您可以從 Actions 控制台部署 Webhook。

如要部署 Webhook，請按照下列步驟操作：

1. 在 Actions 控制台中，按一下導覽列中的「開發」。
2. 按一下導覽列中的「Webhook」分頁標籤。
3. 按一下「Deploy Fulfillment」(部署完成)。

注意：按一下「Deploy Fulfillment」後，畫面上會顯示「需要帳單帳戶」訊息。如要部署 Webhook，您必須在 Google Cloud 中將帳單帳戶與專案建立關聯。如要設定帳單帳戶，請按一下「開啟 Cloud 控制台」，然後按照步驟連結帳單帳戶。您可以在本程式碼研究室結束時刪除這個專案，以免產生費用。詳情請參閱「[清除所用資源](#)」一節。

Cloud Functions 可能需要幾分鐘的時間，才能佈建及部署完成。畫面上應會顯示「Cloud Function deployment in progress...」(正在部署 Cloud Function...)訊息。程式碼成功部署後，訊息會更新為「您的 Cloud Functions 部署作業已完成」。

在模擬工具中測試

此時，您的動作已連結至網頁應用程式。您可以使用 Actions 控制台模擬器，確保在叫用動作時顯示網頁應用程式。

如要測試動作，請按照下列步驟操作：

1. 在 Actions 控制台導覽列中，按一下「測試」。
2. 在「Input」欄位中輸入 `Talk to Snow Pal sample`，然後按下 `Enter` 鍵。
3. 在「Input」欄位中輸入 `Yes`，然後按下 `Enter` 鍵。或者點選「開始遊戲」。

Start typing or click the mic and speak

Suggestions

cancel

Snow Pal sample



Smart Displa...English (United Sta...Googleplex...State EditorSettings

Reset TestView logs in Google Cloud Platform

Talk to Snow Pal sample

Got it. Getting the test version of Snow Pal sample.
Welcome to Snow Pal! Would you like to play again or quit?

Yes

Try guessing a letter in the word, or guess the entire word if you're feeling confident!

您尚未設定猜測字母或單字的邏輯，因此如果猜測單字或字母，系統會回覆「...Incorrect. 繼續努力！看來我們需要新增更多功能，才能讓這項功能正常運作。」

5. 瞭解互動式畫布基礎架構

在本專案中，功能分為三個主要元件：對話動作、網頁應用程式和 Webhook。請注意，這只是設定動作的一種模型，我們以這種方式整理，是為了突顯互動式畫布的核心功能。

以下各節將詳細說明對話式動作、Webhook 和網頁應用程式如何搭配運作，以及其他重要的互動式畫布元素。

對話動作

動作的對話式動作元件會負責辨識及處理使用者輸入內容，並將內容傳送至適當的場景，建構給使用者的回應。舉例來說，如果使用者在 *Snow Pal* 遊戲中說出「I guess the letter e」，對話式動作會將該字母擷取為意圖參數，並傳遞至適當的遊戲邏輯，判斷猜測是否正確，並據此更新網頁應用程式。您可以在 Actions 管理中心的網頁式 IDE「Actions Builder」中，查看及修改這項對話邏輯。以下螢幕截圖顯示 Actions Builder 中的對話動作部分內容：

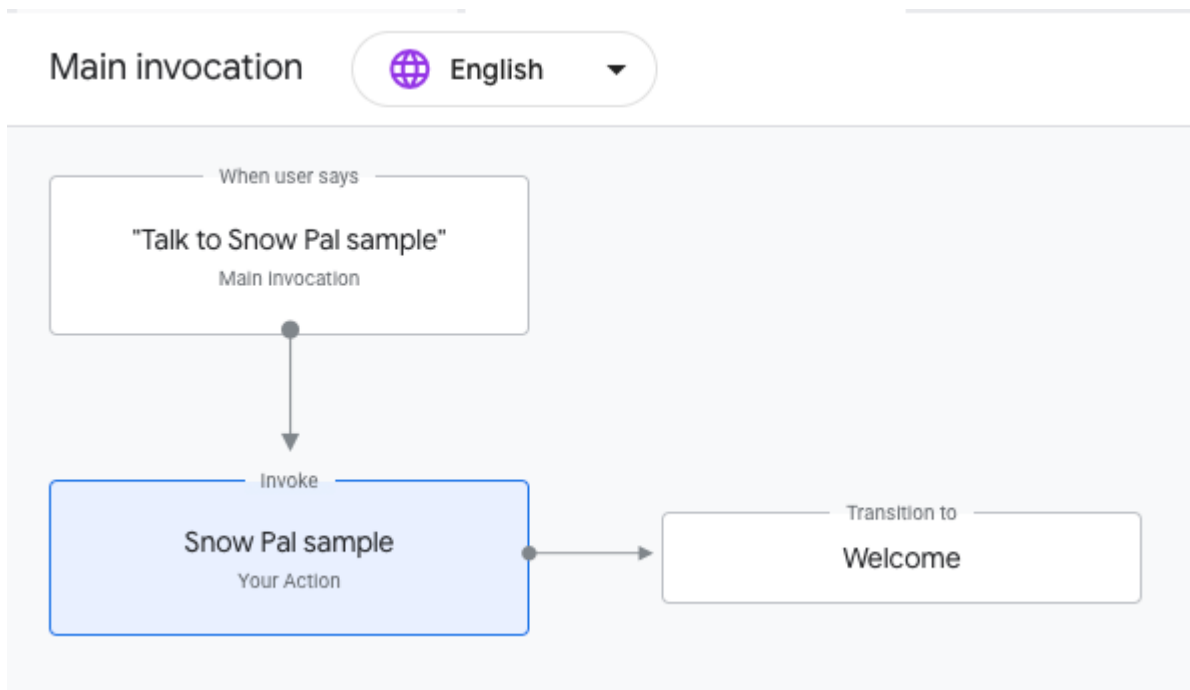


圖 4. Actions Builder 中的 `Main invocation` 視覺化呈現。

這張螢幕截圖顯示動作的 `Main invocation`，使用者說出「Ok Google，與 Snow Pal 範例對話」等片語時，就會比對這個名稱。使用者叫用動作時，`Main invocation` 會傳送含有 `canvas` 回應的提示，其中包含網頁應用程式的網址。

關鍵字詞：

- **Canvas**：包含網頁應用程式網址和/或要傳遞資料的回應。您可以在 Webhook 或 Actions Builder 中設定 Canvas 回應。詳情請參閱「[Canvas 提示](#)」。

Action 的第一個 `canvas` 回應必須包含網頁應用程式網址。這項回應會告知 Google 助理，要在使用者裝置上顯示該網址的網頁應用程式。Actions Builder 中的其他 `canvas` 回應可以包含設為 `true` 的 `send_state_data_to_canvas_app` 欄位。當意圖相符時，這個欄位會將意圖名稱和任何參數值傳送至網頁應用程式，而網頁應用程式會根據使用者輸入的資料更新。

Webhook

在本程式碼研究室中，Webhook 包含遊戲邏輯 (您可以將 Webhook 視為遊戲伺服器)。遊戲邏輯包含判斷使用者猜對或猜錯、使用者是否獲勝或落敗，以及根據結果建構回覆等元素。您可以在動作建構工具中修改 Webhook。

當動作需要執行遊戲邏輯時，動作建構工具會呼叫 Webhook。舉例來說，Game 場景中的 guess 意圖會對 guess 處理常式發出 Webhook 呼叫，然後執行邏輯，判斷使用者的猜測是否正確。Webhook 可以在對應至網頁應用程式檔案的處理常式中包含 Canvas 回應，並適當更新網頁。

網頁應用程式

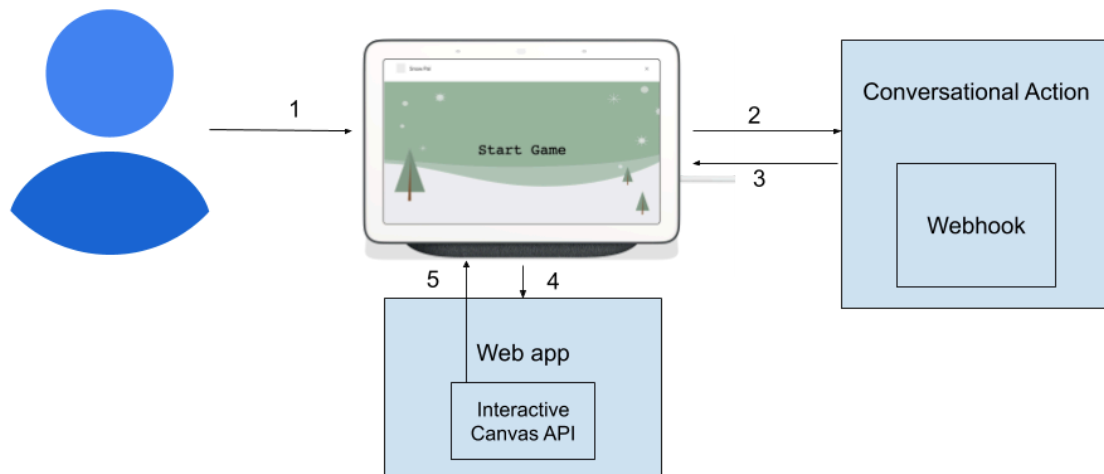


圖 5. 以視覺化方式呈現互動式畫布動作中，對話動作、Webhook 和網頁應用程式之間的互動。

網頁應用程式檔案包含網頁應用程式的視覺效果和動畫程式碼，以及根據使用者輸入內容更新網頁應用程式的邏輯。您可以在文字編輯器中修改網頁應用程式檔案，然後使用 Firebase CLI 部署這些變更。

在對話動作和網頁應用程式之間通訊

您必須啟用對話動作與網頁應用程式之間的通訊，網頁應用程式才能根據使用者輸入內容更新。舉例來說，如果使用者說「我猜是字母 f」，

對話動作需要向網頁應用程式提供字母 *f*，網頁應用程式才能據此更新。如要將使用者的輸入內容從對話式動作傳遞至網頁應用程式，您必須載入 Interactive Canvas API。

重要字詞：

- **互動式畫布 API**：您在網頁應用程式中加入的 JavaScript API，可讓網頁應用程式與對話動作進行通訊。

這個 API 的指令碼包含在 `/public/index.html` 中，這是「Snow Pal」遊戲的主要 HTML 檔案。這個檔案會定義 UI 的外觀，並在多個指令碼中載入：

index.html


```
<!-- Load Assistant Interactive Canvas API -->
<script type="text/javascript"
src="https://www.gstatic.com/assistant/interactivecanvas/api/interactive_canvas.min.js">
</script>
```

如要根據使用者輸入內容更新網頁應用程式，您也必須在網頁應用程式檔案中註冊及設定 *回呼*。網路應用程式可透過回呼功能，回應對話式動作傳送的資訊或要求。

在 `/public/js/action.js` 中，有一個預先設定的類別 `Action`，用於宣告及註冊回呼。`Action` 類別是 `Interactive Canvas API` 的包裝函式。使用 `scene.js` 中的 `create()` 函式建立網頁應用程式時，系統會建立新的 `Action` 例項並呼叫 `setCallbacks()`，如下方程式碼片段所示：

scene.js

```
// Set Assistant at game level.
this.assistant = new Action(this);
// Call setCallbacks to register Assistant Action callbacks.
this.assistant.setCallbacks();
```

`setCallbacks()` 函式定義於 `/public/js/action.js` 的 `Action` 類別中。這項函式會宣告回呼，並在建立遊戲時向 `Interactive Canvas API` 註冊回呼：

```
setCallbacks() {
  // Declare the Interactive Canvas Action callbacks.
  const callbacks = {
    onUpdate: (data) => {
      ...
    }
  }
  // Called by the Interactive Canvas web app once web app has loaded to
  // register callbacks.
  this.canvas.ready(callbacks);
}
```

`setCallbacks()` 函式會宣告 `onUpdate()` 回呼，每次傳送 `Canvas` 回應時都會呼叫該回呼。

重要字詞：

- **`onUpdate()`**：每次傳送 `Canvas` 回應時執行的回呼。`onUpdate()` 可以對應至更新網頁應用程式的自訂邏輯。

下一節說明如何設定這個專案的特定程式碼，將資料從對話式動作傳遞至 Web 應用程式。

根據使用者輸入內容更新網頁應用程式

在本程式碼研究室中，您會使用指令對應更新網頁應用程式，具體做法是根據使用者輸入的內容。舉例來說，當

`start_game` 意圖在 `welcome` 場景中相符時，系統會將提示中包含的 `canvas` 回覆傳送至網頁應用程式。`onUpdate()` 會剖析 `canvas` 回覆中的中繼資料，並呼叫 `START_GAME` 指令，而該指令會呼叫 `scene.js` 中的 `start()` 函式，並更新網頁應用程式以開始新的遊戲工作階段。

`scene.js` 中的 `start()` 函式也會呼叫 `updateCanvasState()` 函式，該函式會使用名為 `setCanvasState()` 的方法，新增 Webhook 可存取的狀態資料。

重要字詞：

- **`setCanvasState()`**：這個方法可讓您將互動式畫布網頁應用程式的狀態資料傳送至 Webhook。網頁應用程式會以 JSON 物件的形式傳送更新後的狀態。

每個指令的結尾都會呼叫 `updateCanvasState()` 方法 (您會在整個程式碼研究室中新增這些指令)，並更新網頁應用程式的狀態。每次呼叫 `updateCanvasState()` 時，系統都會根據目前狀態更新 `displayedWord` 和 `incorrectGuesses` 的值：

scene.js

```
...
updateCanvasState() {
  window.interactiveCanvas.setCanvasState({
    correctWord: this.word.text,
    displayedWord: this.word.displayText.text,
    incorrectGuesses: this.incorrectGuesses,
  });
}
```

更新後的狀態隨即會用於下一個對話回合。您可以在 Webhook 中透過 `conv.context.canvas.state` 存取此狀態，如下程式碼片段所示：

index.js

```
...
let displayedWord = conv.context.canvas.state.displayedWord;
...
```

6. 新增猜測功能

在本節中，您要為動作新增 `guess` 功能，讓使用者猜測字詞中的字母或字詞本身。

對話動作

在「在模擬器中測試」部分，您收到「It looks like we need to add more functionality to have this work properly.」(我們似乎需要新增更多功能，才能正常運作。)回覆。現在，您可以在 Actions 控制台中刪除該提示，這樣系統只會呼叫 Webhook (在 `Game` 場景中，`guess` 意圖已設定為在相符時呼叫 Webhook)。

如要在比對到 `guess` 意圖時移除靜態提示，請按照下列步驟操作：

1. 在 Actions 控制台中，按一下導覽列中的「場景」。
2. 按一下「遊戲」即可前往 `Game` 場景。
3. 按一下「使用者意圖處理」下方的「當猜測結果相符時」。清除「傳送提示」即可移除提示。
4. 按一下 [儲存]。

Webhook

在本節中，您將更新 Webhook，將使用者猜對或猜錯的答案對應至網路應用程式檔案中的邏輯，然後據此更新網路應用程式。系統已在 Webhook 中為您設定 `guess` 意圖處理常式，因此您只需要將 `Canvas` 回覆新增至這個意圖，即可觸發更新網頁應用程式的邏輯。

如要更新 Webhook，請按照下列步驟操作：

1. 在 Actions 控制台中，按一下導覽列中的「Webhook」。
2. 在 `guess` 處理常式下方的 `index.js` 中新增下列程式碼：

注意：在本程式碼研究室中，您新增的程式碼片段會分成依字母順序排列的區段。您修改的檔案會包含註解，指出應在何處新增每個程式碼片段 (例如 `// Add section F } here`)。請在註解行後方新增對應的程式碼片段。

`index.js` (A 部分)：

```
// Add Section A `conv.add(new Canvas({` content here.
conv.add(new Canvas({
  data: {
    command: 'CORRECT_ANSWER',
    displayedWord: displayedWord
  },
}));
```

`index.js` (B 部分)：

```
// Add Section B `conv.add(new Canvas({` content here.
conv.add(new Canvas({
  data: {
    command: 'INCORRECT_ANSWER',
  },
}));
```

3. 按一下「儲存出貨資訊」。
4. 按一下「Deploy Fulfillment」（部署完成）。部署完成後，訊息會顯示「您的 Cloud 函式部署作業已是最新版本」。

網頁應用程式

您現在可以設定網頁應用程式，處理 `CORRECT_ANSWER` 和 `INCORRECT_ANSWER` 指令。

1. 在文字編輯器中開啟 `public/js/action.js`。
2. 更新網頁應用程式，以處理 `CORRECT_ANSWER` 和 `INCORRECT_ANSWER` 指令：

action.js (C 部分)：

```
// Add Section C `CORRECT_ANSWER: (params) => {` content here.
CORRECT_ANSWER: (params) => {
  this.gameScene.correctAnswer(params);
},
INCORRECT_ANSWER: (params) => {
  this.gameScene.incorrectAnswer();
},
```

3. 執行下列指令來更新 Web 應用程式：

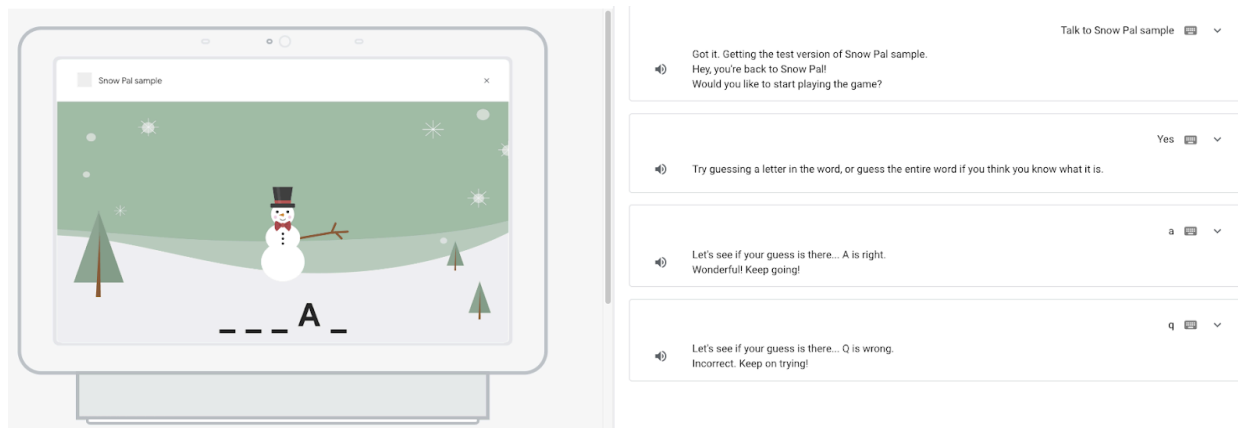
```
firebase deploy --project {PROJECT_ID} --only hosting
```

在模擬工具中測試動作

此時，動作可以辨識猜測是否正確，並據此更新網路應用程式。

如要測試動作，請按照下列步驟操作：

1. 按一下導覽列中的「測試」。
2. 在「Input」欄位中輸入 `Talk to Snow Pal sample`，然後按下 `Enter` 鍵。
3. 在「Input」欄位中輸入 `Yes`，然後按下 `Enter` 鍵。或者，按一下「是」按鈕。
4. 在「輸入」欄位中輸入要猜的字母，然後按下 `Enter`。



瞭解程式碼

在上一節中，您新增了程式碼，讓使用者可以在遊戲中猜測字母，並在字詞或雪人中看到這些猜測結果。大致來說，當 `guess` 意圖相符時，您會在 Actions Builder 中發出 Webhook 呼叫，將資料傳遞至網頁應用程式，以便相應更新。舉例來說，如果使用者在「Snow Pal」遊戲中猜對字詞中的某個字母，網頁應用程式就會更新，在字詞的正確位置顯示該字母。

如果動作使用互動式畫布，資料從 Webhook 傳遞至網頁應用程式的一般流程如下：

1. 使用者的輸入內容符合包含 `Canvas` 回應的意圖。
2. 對話動作或 Webhook 會傳送 `Canvas` 回應，觸發 `onUpdate()` 回呼。
3. `onUpdate()` 回呼會對應至自訂邏輯，據此更新網頁應用程式。

重要字詞：

- `onUpdate()`：每當完成動作傳送 `Canvas` 回應時，系統就會執行這個回呼。• `onUpdate()` 可對應至更新網頁應用程式的自訂邏輯。

就這個特定專案而言，程式碼的運作方式如下：

1. 當使用者符合 `guess` 意圖時，動作建構工具會從使用者的輸入內容中擷取字母做為參數。
2. Actions Builder 會在 Webhook 中呼叫 `guess` 處理常式，其中包含判斷使用者猜測的字母是否出現在單字中的邏輯。
3. `guess` 處理常式包含兩個 `Canvas` 回應，分別在字母正確和不正確時執行。每個 `Canvas` 回應都會將適當的資料 (`CORRECT_ANSWER` 或 `INCORRECT_ANSWER` 指令) 傳遞至 Web 應用程式。
4. `Canvas` 回應的 `data` 欄位中包含的資料會傳遞至 `action.js` 中的 `onUpdate()` 方法。 `onUpdate()` 會在 `scene.js` 的指令對應中呼叫適當的指令。
5. 命令對應會對應至 `scene.js` 中的 `correctAnswer()` 和 `incorrectAnswer()` 函式。這些函式會適當更新網頁應用程式，反映使用者的猜測，並呼叫 `setCanvasState()`，將網頁應用程式的狀態資料傳送至 Webhook。

步驟 7 · 約 0 分鐘

7. 新增成交/未成交功能

在本節中，您要將輸贏功能新增至動作，包括判斷使用者輸贏的邏輯，以及根據使用者結果更新網頁應用程式圖片的邏輯。

對話動作

處理使用者輸贏遊戲的功能會在 `guess` 意圖中設定，因此您不必在 Actions Builder 中進行任何額外設定。

Webhook

在本節中，您將更新 Webhook，加入處理使用者輸贏的邏輯，並對應至 Web 應用程式邏輯，以便更新遊戲，顯示適當的輸贏畫面。

如要更新 Webhook，請按照下列步驟操作：

1. 在 Actions 控制台中，按一下導覽列中的「Webhook」。
2. 在 `guess` 處理常式下方的 `index.js` 中新增下列程式碼：

index.js (第 D 節)：

```
// Add Section D `if (userHasWon)` content here.
if (userHasWon) {
  conv.add(`<speak>Let's see if your guess is there...<br>
    time='2500ms'/> ${guess} is right. That spells ${correctWord}!
    ${randomArrayItem(WIN_RESPONSES)}</speak>`);
  conv.add(new Canvas({
    data: {
      command: 'WIN_GAME',
      displayedWord: displayedWord
    },
  }));
  conv.add(`<speak>${PLAY_AGAIN_INSTRUCTIONS}</speak>`);
} else {
```

index.js (第 E 節)：

注意：在本節中，您會新增右括號來關閉程式碼區塊。

```
// Add Section E `}` here.
}
```

index.js (第 F 節)：

```
// Add Section F `Check if the user has exceeded the maximum` content here.
// Check if the user has exceeded the maximum amount of max guesses allowed.
const userHasLost = conv.context.canvas.state.incorrectGuesses + 1 >=
MAX_INCORRECT_GUESSES;
if (userHasLost) {
  conv.add(`<speak>Let's see if your guess is there...<break
time='2500ms'/> ${guess} is wrong. Sorry you lost. The word is ${correctWord}!
</speak>`);
  conv.add(new Canvas({
    data: {
      command: 'LOSE_GAME',
    },
  }));
  conv.add(`<speak>${PLAY_AGAIN_INSTRUCTIONS}</speak>`);
} else {
```

index.js (第 G 節)：

注意：在本節中，您會新增右括號來關閉程式碼區塊。

```
// Add Section G `}` here.
}
```

3. 按一下「儲存出貨資訊」。
4. 按一下「Deploy Fulfillment」(部署完成)。部署完成後，訊息會顯示「您的 Cloud 函式部署作業已是最新版本」。

您在此新增了兩個 `Canvas` 回應，並使用 `WIN_GAME` 和 `LOSE_GAME` 指令，處理使用者贏得或輸掉遊戲的情況。在下一節中，您將新增功能，根據使用者是否獲勝來更新網頁應用程式。

網頁應用程式

您現在可以根據使用者輸贏，設定網頁應用程式的更新方式。如要更新網頁應用程式，請按照下列步驟操作：

1. 在文字編輯器中開啟 `public/js/action.js`。
2. 更新網頁應用程式，以處理 `WIN_GAME` 和 `LOSE_GAME` 指令：

action.js (第 H 節)：

```
// Add Section H `WIN_GAME: (params) => {` content here.
WIN_GAME: (params) => {
  this.gameScene.winGame(params);
},
LOSE_GAME: (params) => {
  this.gameScene.loseGame();
},
```

3. 執行下列指令來更新 Web 應用程式：

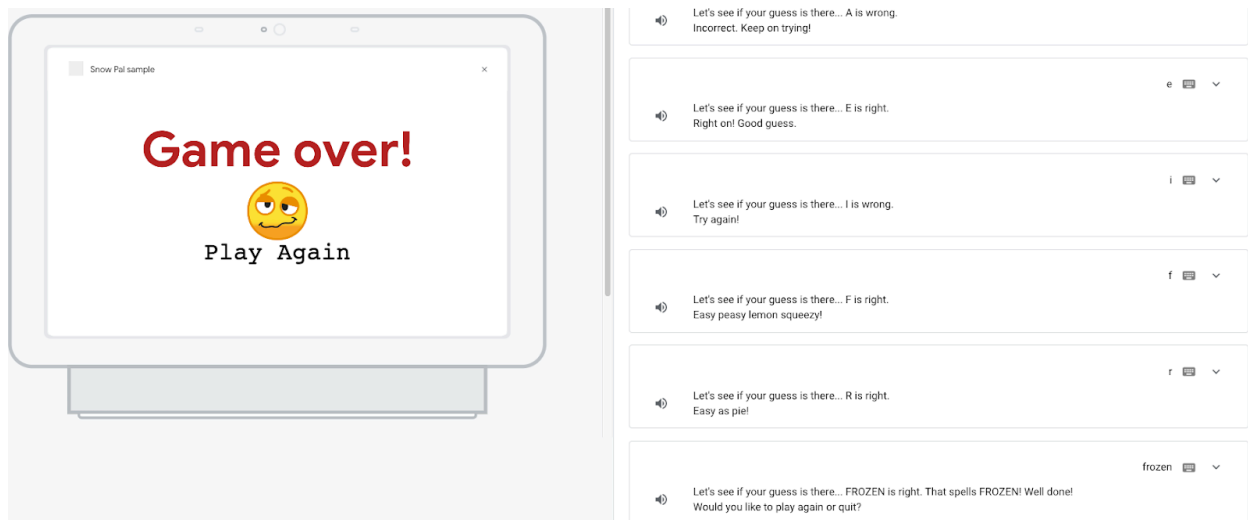
```
firebase deploy --project {PROJECT_ID} --only hosting
```

在模擬工具中測試動作

此時，動作可以處理使用者贏得或輸掉遊戲的情況，並根據每種結果顯示適當的畫面。

如要測試動作，請按照下列步驟操作：

1. 在 Actions 控制台導覽列中，按一下「測試」。
2. 在「Input」欄位中輸入 `Talk to Snow Pal sample`，然後按下 `Enter` 鍵。
3. 在「Input」欄位中輸入 `Yes`，然後按下 `Enter` 鍵。你也可以點選「開始遊戲」按鈕。
4. 猜測字母和單字，直到贏得遊戲或輸掉為止。



如果你要求再次播放，系統會顯示訊息，指出再次播放所需的功能尚未新增。您將在下一節中新增這項功能。

瞭解程式碼

輸贏功能與猜測功能運作方式相同：使用者符合 `guess` 意圖，而您的 Webhook 會評估使用者的猜測。如果使用者猜對，程式碼會檢查使用者是否獲勝；如果是，程式碼會將 `WIN_GAME` 指令傳送至網頁應用程式。如果使用者猜錯，程式碼會檢查使用者是否輸了；如果是，程式碼會將 `LOSE_GAME` 指令傳送至網頁應用程式。這些指令會觸發 `scene.js` 中的 `winGame()` 和 `loseGame()` 函式，更新網頁應用程式以顯示輸贏畫面，並更新遊戲狀態。

8. 新增重新播放功能

在本節中，您將新增功能，讓使用者說出「Play again」(再玩一次)，或點選網頁應用程式中的「Play Again」(再玩一次) 按鈕，即可開始新遊戲。在 Actions Builder 中修改 `play_again` 意圖，傳送 `canvas` 回應來適當更新網頁應用程式，並新增邏輯，在使用者點選「Play Again」(再玩一次) 按鈕時觸發 `play_again` 意圖。

對話動作

在上一節測試動作時，如果您嘗試再次玩遊戲，系統會顯示以下提示：「這很棒，但我們會在後續章節中建構這項功能。目前只要重設動作即可。」現在可以刪除這個提示，並換成在使用者要求其他遊戲時回覆的提示(「好的，這是另一款遊戲！」)，並加入 `canvas` 回覆，觸發網頁應用程式啟動新遊戲。

如要在使用者想再次播放時更新提示，請按照下列步驟操作：

1. 在 Actions 控制台中，按一下「場景」下拉式選單。
2. 按一下「Game」場景。
3. 按一下「使用者意圖處理」下方的「當系統比對到 `play_again` 時」。
4. 將提示詞換成下列文字：

```
candidates:
  - first_simple:
      variants:
        - speech: 'Okay, here's another game!'
      canvas:
        sendStateDataToCanvasApp: true
```

5. 按一下 [儲存]。

Webhook

在本程式碼研究室中，Webhook 會管理遊戲邏輯。由於「再玩一次」功能不需要任何邏輯驗證，因此您不需要呼叫 Webhook，而是可以直接從 Actions Builder 傳送 `canvas` 回應，將必要資料傳遞至網頁應用程式(您已在上一節中設定)。

網頁應用程式

現在可以修改網頁應用程式檔案，在使用者要求再次播放時適當更新。如要新增這項功能，請按照下列步驟操作：

1. 在文字編輯器中開啟 `public/js/action.js`。
2. 更新網頁應用程式，處理 `PLAY_AGAIN` 指令：

action.js (第 1 節)：

```
// Add Section I `PLAY_AGAIN: (params) => {` content here.
PLAY_AGAIN: (params) => {
  this.gameScene.start();
},
```

3. 在文字編輯器中開啟 `public/js/scene.js`。
4. 更新網頁應用程式，讓使用者點選「Play Again」按鈕時，啟動新的遊戲工作階段：

scene.js (第 J 節)：

```
// Add Section J `sendTextQuery` content here.  
window.interactiveCanvas.sendTextQuery('Play again');
```

5. 執行下列指令來更新 Web 應用程式：

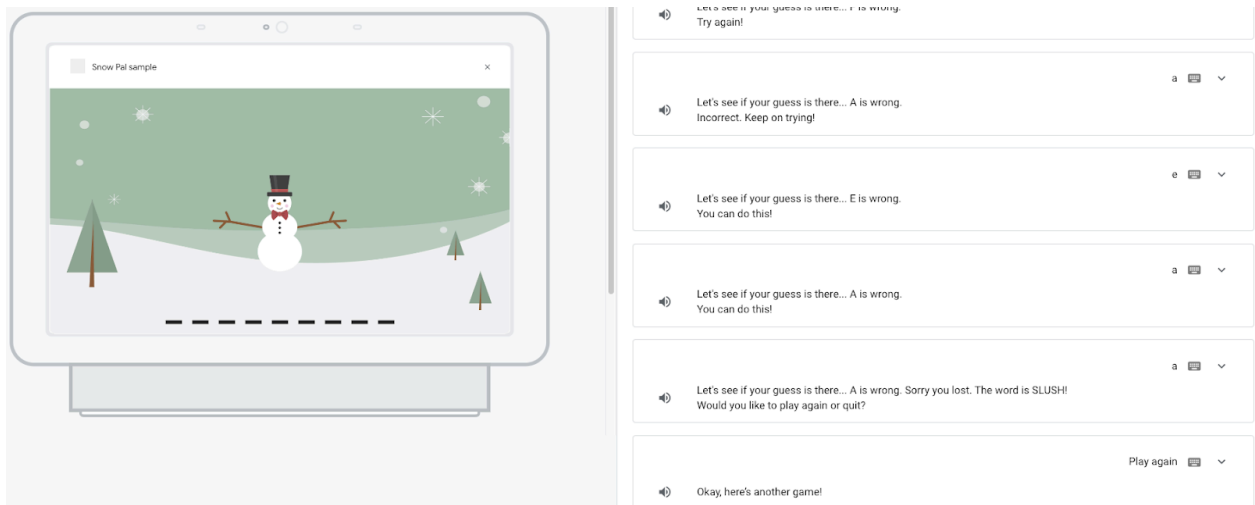
```
firebase deploy --project {PROJECT_ID} --only hosting
```

在模擬工具中測試動作

現在，使用者說出「Play again」(再玩一次) 或點選「Play Again」(再玩一次) 按鈕時，你的動作就能啟動新的遊戲工作階段。

如要測試動作，請按照下列步驟操作：

1. 按一下導覽列中的「測試」。
2. 在「Input」欄位中輸入 `Talk to Snow Pal sample`，然後按下 `Enter` 鍵。
3. 在「Input」欄位中輸入 `yes`，然後按下 `Enter` 鍵。你也可以點選「開始遊戲」按鈕。
4. 猜測字母和單字，直到贏得遊戲或輸掉為止。
5. 在「Input」欄位中輸入 `Play again`，然後按下 `Enter` 鍵。你也可以點選「再次播放」按鈕。



瞭解程式碼

測試動作時，你可以透過語音輸入 (「Play again」) 或觸控輸入 (按一下「Play again」按鈕) 啟動新遊戲。

如果是語音輸入選項，當使用者說「再玩一次」或類似的內容時，系統會比對出 `play_again` 意圖，並將提示 (「好的，再來一場！」) 加入提示佇列。提示中包含的 `canvas` 回應會將意圖名稱和其他中繼資料傳送至 Web 應用程式。意圖名稱會傳遞至 `onUpdate()` 回呼，將對應的指令 `PLAY_AGAIN` 對應至 `action.js` 中的指令對應。`PLAY_AGAIN` 指令會觸發 `scene.js` 中的 `start()` 函式，並使用新的遊戲工作階段更新網頁應用程式。

如要使用觸控輸入選項，請使用 `sendTextQuery()` (Interactive Canvas API)，透過觸控輸入觸發意圖，讓按鈕正常運作。

重要字詞：

- `sendTextQuery()`：這個方法會將文字查詢傳送至對話動作，以程式輔助方式叫用意圖。

在本程式碼研究室中，您會使用 `sendTextQuery()`，在使用者點選「Play again」按鈕時叫用 `play_again` 意圖。`Play again` 引數會比對 `play_again` 意圖中的訓練詞組，並觸發該意圖，就像使用者說出「Play again」(再播放一次) 一樣。`play_again` 意圖接著會觸發邏輯，更新網頁應用程式並開始新的遊戲工作階段。

9. 更新 `PLAY_GAME` 內建意圖

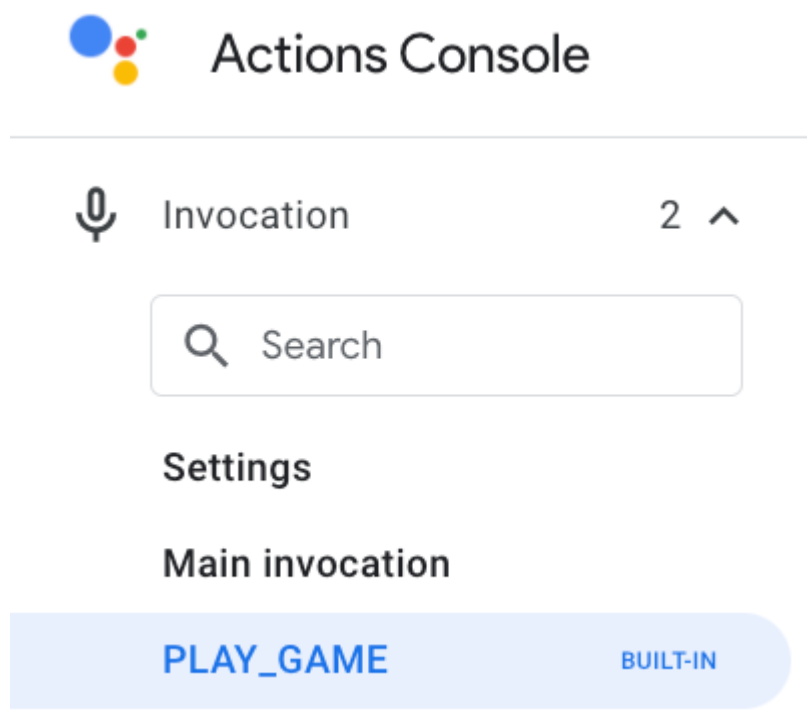
在本節中，您將更新 `PLAY_GAME` 內建意圖。

重要字詞：

- 內建意圖：**您可以指定不重複的 ID，告知 Google 助理您的動作可滿足特定類別的使用者要求。

使用者發出一般要求 (例如「我想玩遊戲」) 時，`PLAY_GAME` 內建意圖可讓使用者叫用動作。

原始碼包含 `PLAY_GAME` 內建意圖，位於 `/sdk/custom/global/actions.intent.PLAY_GAME.yaml`。這會反映在控制台的「Invocation」部分，顯示為 `PLAY_GAME`，如下列螢幕截圖所示：



如要讓使用者透過這個內建意圖叫用動作，您需要在 `PLAY_GAME` 內建意圖中新增含有網頁應用程式網址的 `canvas` 回應。步驟如下：

- 在 Actions 控制台中，按一下導覽列中的「`PLAY_GAME`」`PLAY_GAME`。
- 更新提示，加入網路應用程式網址，如以下程式碼片段所示：

```
candidates:
- canvas:
  url: 'https://<PROJECT_ID>.web.app'
```

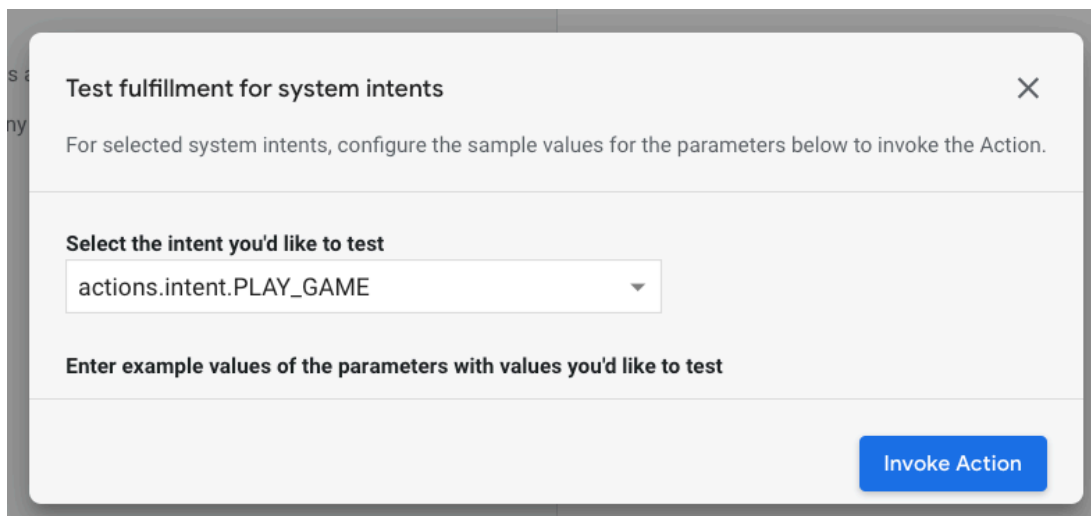
- 按一下 [儲存]。

在模擬工具中測試動作

您的動作現在支援 `PLAY_GAME` 內建意圖。

如要測試動作，請按照下列步驟操作：

1. 按一下導覽列中的「測試」。
2. 按一下「測試內建意圖處理」。
3. 按一下「Invoke Action」 (叫用動作)。



Test fulfillment for system intents ✕

For selected system intents, configure the sample values for the parameters below to invoke the Action.

Select the intent you'd like to test

actions.intent.PLAY_GAME ▼

Enter example values of the parameters with values you'd like to test

[Invoke Action](#)

您應在模擬器中叫用動作。

10. 附錄：排解 Interactive Canvas 動作問題

在本節中，您將瞭解如何偵錯無法正常運作的互動式畫布動作。Snow Pal 專案已預先封裝偵錯重疊畫面，您可以啟用該畫面。如以下螢幕截圖所示，疊加層會將所有 `console.log()` 和 `console.error()` 輸出內容顯示在螢幕右下角：



如要啟用這個疊加層，請開啟 `/public/css/main.css` 檔案並註解 `display: none !important;` 行，如以下程式碼片段所示：

main.css

```
.debug {  
  display: flex;  
  flex-direction: column;  
  
  /* Comment below to view debug overlay */  
  /* display: none !important; */  
  
  width: 500px;  
  height: 150px;  
  right: 0;  
  bottom: 0;  
  position: absolute;  
}
```

清除專案所用資源 (建議)

為避免產生可能的費用，建議您移除不打算使用的專案。如要刪除您在本程式碼研究室中建立的專案，請按照下列步驟操作：

1. 如要刪除 Google Cloud 專案和資源，請完成「[關閉 \(刪除\) 專案](#)」一節列出的步驟。

注意：請務必在 Google Cloud Platform 設定頁面中選取要刪除的正確專案。

2. 選用：如要立即從 Actions 管理中心移除專案，請完成「[刪除專案](#)」一節列出的步驟。如果未完成這個步驟，系統會在約 30 天後自動移除專案。

11. 恭喜！

您已完成互動式畫布入門程式碼研究室，現在具備建構專屬互動式畫布動作的必要技能。

目前所學內容

- 如何建構、部署及測試 Interactive Canvas 動作
- 如何使用 `Canvas` 回覆更新網路應用程式
- 如何使用不同方法強化動作，例如 `sendTextQuery()` 和 `setCanvasState()`
- 如何對動作進行偵錯

瞭解詳情

如要進一步瞭解互動式畫布，請參閱下列資源：

- [Interactive Canvas 說明文件](#)：Actions on Google 官方的 Interactive Canvas 說明文件。
- [互動式畫布範例](#)：簡單的互動式畫布遊戲程式碼，可讓您旋轉三角形並修改顏色。
- [遊戲入口網站](#)：在 Google 助理上建構遊戲的指南。

意見回饋問卷調查

離開前，請填寫簡短的[問卷調查](#)，分享你的體驗。
